

EXP.NO: 4	System Calls
DATE:	

AIM:

To study and implement basic UNIX system calls such as process creation, execution, process identification, and directory handling using C programs.

ALGORITHM:

1. Start the program and include necessary headers (stdio.h, unistd.h, dirent.h).
2. Identify Process: Use getpid() and getppid() to fetch ID numbers from the Kernel.
3. Process Creation: Invoke fork(). Branch execution based on return value (0 for child, positive for parent).
4. Process Replacement: Use exec() family functions to replace current process image with a new executable.
5. Directory Access: Initialize a DIR pointer with opendir().
6. Read Entries: Use a while loop with readdir() to iterate through file entries until NULL is reached.
7. Cleanup & Stop: Close directory streams and terminate the program.

CODE:

Process Execution:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec()\n");
    execl("/bin/ls", "ls", NULL);
    printf("After exec()\n"); // This line will not execute on success
    return 0;
}
```

Process Creation:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    if (fork() == 0) {
```

```

    printf("This is CHILD process\n");
} else {
    printf("This is PARENT process\n");
}
printf("PID: %d\n", getpid());
return 0;
}

```

Process Identification:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());
    return 0;
}

```

Directory Handling:

```

#include <stdio.h>
#include <dirent.h>

int main() {
    DIR *dir;
    struct dirent *entry;
    dir = opendir(".");
    if (dir == NULL) {
        printf("Cannot open directory\n");
        return 1;
    }
    printf("Files in directory:\n");
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }
    closedir(dir);
    return 0;
}

```

OUTPUT:

```
navdeep@localhost:~/os_exp/sys
[navdeep@localhost os_exp]$ ls
awk  shell  sys
[navdeep@localhost os_exp]$ cd sys
[navdeep@localhost sys]$ touch pe.c
[navdeep@localhost sys]$ touch pc.c
[navdeep@localhost sys]$ touch pi.c
[navdeep@localhost sys]$ touch dh.c
[navdeep@localhost sys]$ ls
dh.c  pc.c  pe.c  pi.c
[navdeep@localhost sys]$ gcc pe.c -o pe
[navdeep@localhost sys]$ ./pe
Before exec()
dh.c  pc.c  pe  pe.c  pi.c
[navdeep@localhost sys]$ gcc pc.c -o pc
[navdeep@localhost sys]$ ./pc
This is PARENT process
PID: 26472
This is CHILD process
PID: 26473
[navdeep@localhost sys]$ gcc pi.c -o pi
[navdeep@localhost sys]$ ./pi
PID: 26568
PPID: 8219
[navdeep@localhost sys]$ gcc dh.c -o dh
[navdeep@localhost sys]$ ./dh
Files in directory:
.
..
pe.c
pc.c
pi.c
dh.c
pe
pc
pi
dh
[navdeep@localhost sys]$
```

RESULT:

Thus, the basic UNIX system calls such as `fork()`, `exec()`, `getpid()`, and directory handling functions like `opendir()` and `readdir()` were successfully implemented and executed using C programs.